Project Tests

Test overview:

Experiment	Tool	Operations	Target	What It Measures
Exp 1 — Horizontal Scaling	Locust	90 users	5,000 readings/min	P95/P99 ingest latency as ECS tasks scale 1→2→4
Exp 2 — In-memory baseline	Go test script	150 (50 ingest, 50 query, 50 stats)	Live ALB	Write and read latency of the in-memory store — establishes the performance ceiling before any persistence layer is added
Exp 3 — SSE Fan-Out	Locust	550 users	500 concurrent SSE clients	Broadcast ceiling + reconnection behavior under chaos

Load testing is conducted using Locust, driving synthetic Apple Watch biometric data against the live AWS infrastructure via the ALB.

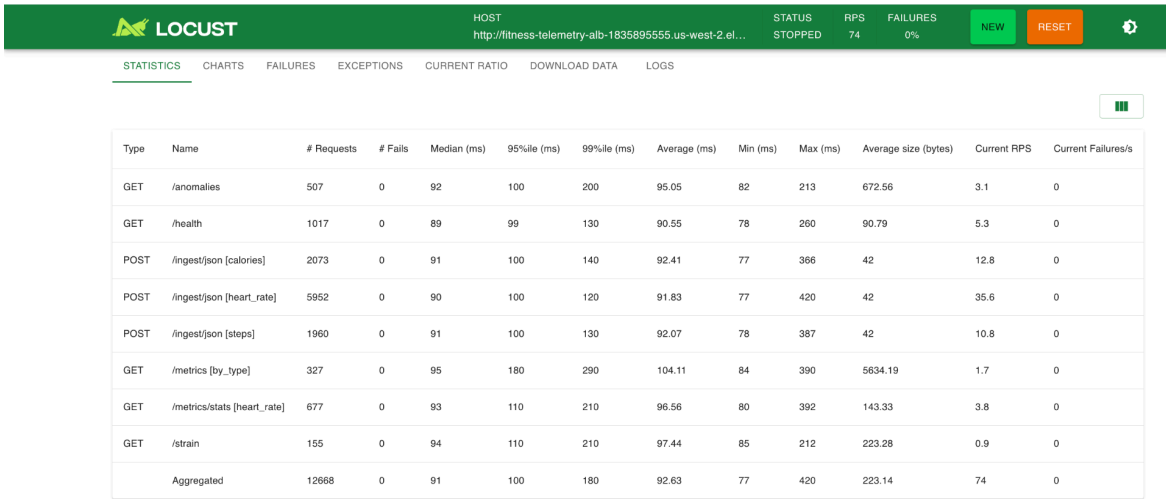
Experiment 1 isolates horizontal scalability by holding ingest load constant at roughly 5,000 readings per minute while incrementally scaling ECS Fargate tasks from 1 to 8, using Terraform to apply each configuration change between runs.

Experiment 2 benchmarked the durability gap in the current architecture. The system uses an in-memory store with no persistence — all biometric data is lost on task restart and diverges across ECS tasks when scaled horizontally. To quantify what a persistence layer would need to match, 150 sequential operations were run against the live ECS deployment via the ALB: 50 ingest batches (5 readings each), 50 metric queries, and 50 stats aggregations.

Experiment 3 stress-tests the SSE fan-out ceiling by holding up to 500 persistent streaming connections open for 20–40 seconds each while a separate user class continuously injects anomaly-triggering heart rate reading with a chaos injection mid-test that kills half the ECS tasks to observe reconnection storm behavior.

Together the three experiments cover the three main failure surfaces of the system: compute scaling, storage durability, and real-time distribution.

Test 1:



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/anomalies	507	0	92	100	200	95.05	82	213	672.56	3.1	0
GET	/health	1017	0	89	99	130	90.55	78	260	90.79	5.3	0
POST	/ingest/json [calories]	2073	0	91	100	140	92.41	77	366	42	12.8	0
POST	/ingest/json [heart_rate]	5952	0	90	100	120	91.83	77	420	42	35.6	0
POST	/ingest/json [steps]	1960	0	91	100	130	92.07	78	387	42	10.8	0
GET	/metrics [by_type]	327	0	95	180	290	104.11	84	390	5634.19	1.7	0
GET	/metrics/stats [heart_rate]	677	0	93	110	210	96.56	80	392	143.33	3.8	0
GET	/strain	155	0	94	110	210	97.44	85	212	223.28	0.9	0
	Aggregated	12668	0	91	100	180	92.63	77	420	223.14	74	0

Figure 1a. Locust statistics of 1 ECS task; 100 users, 10 spawn rate, 2 minutes.

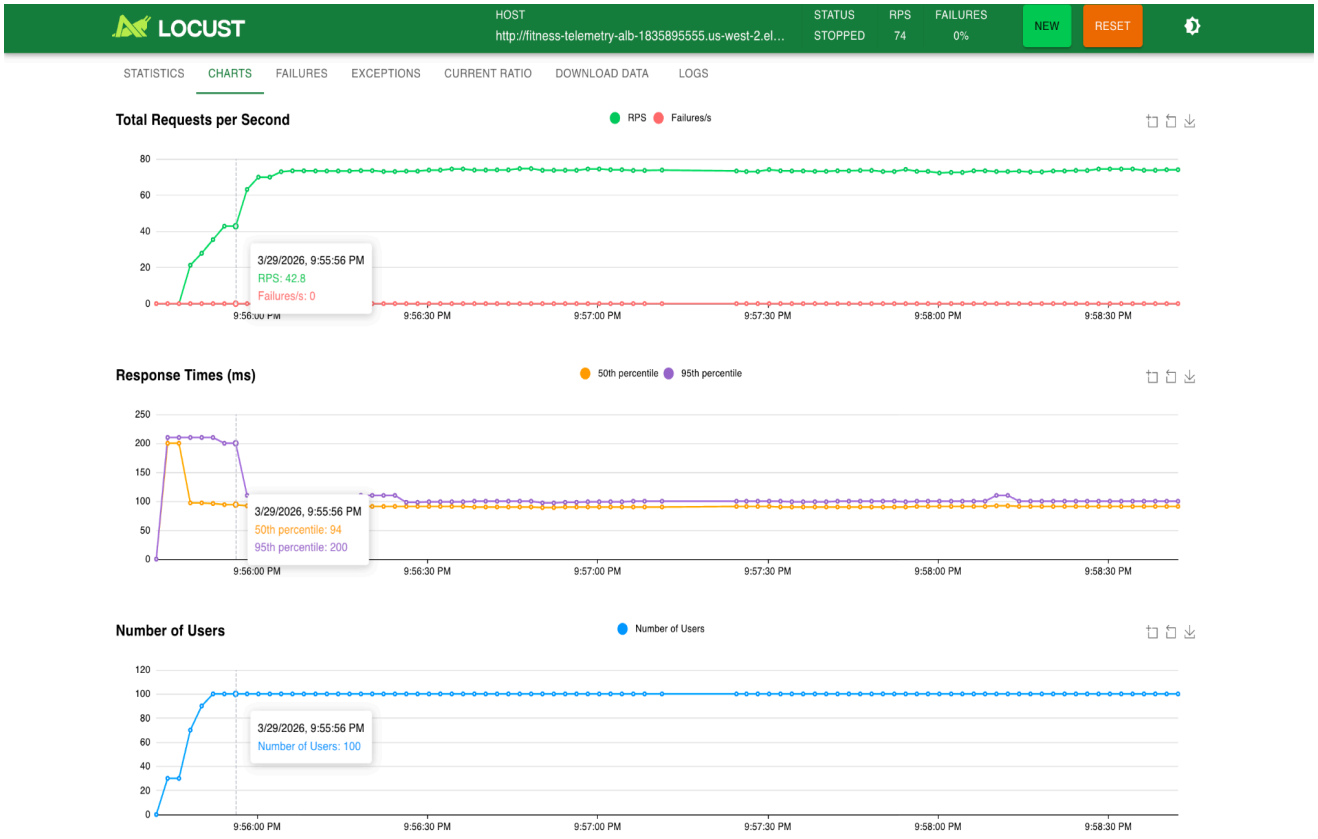
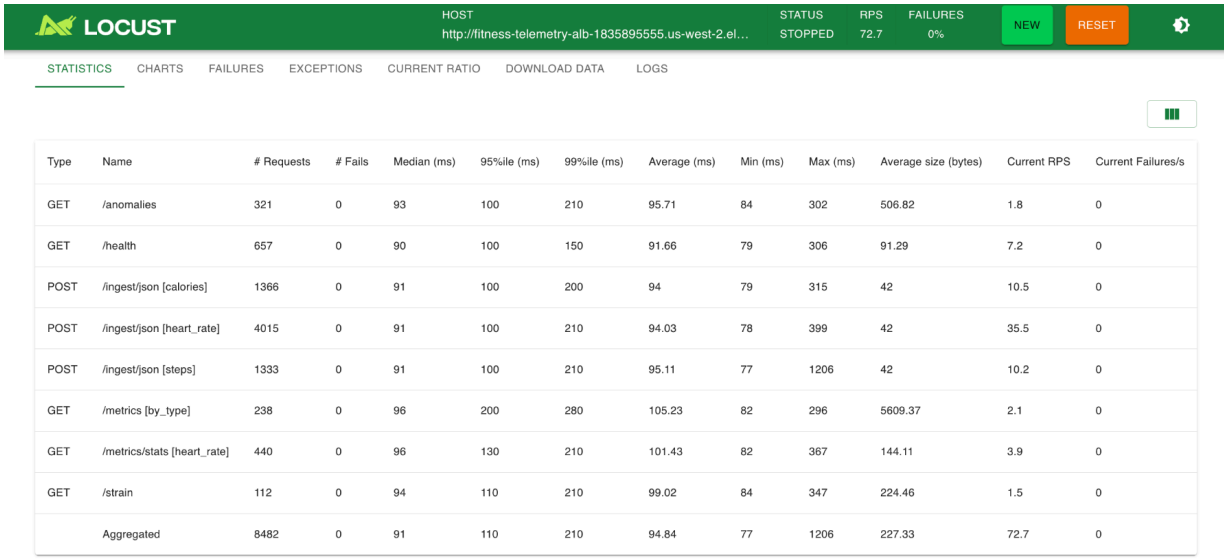


Figure 1b. Locust graphs of 1 ECS task; 100 users, 10 spawn rate, 2 minutes.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/anomalies	321	0	93	100	210	95.71	84	302	506.82	1.8	0
GET	/health	657	0	90	100	150	91.66	79	306	91.29	7.2	0
POST	/ingest/json [calories]	1366	0	91	100	200	94	79	315	42	10.5	0
POST	/ingest/json [heart_rate]	4015	0	91	100	210	94.03	78	399	42	35.5	0
POST	/ingest/json [steps]	1333	0	91	100	210	95.11	77	1206	42	10.2	0
GET	/metrics [by_type]	238	0	96	200	280	105.23	82	296	5609.37	2.1	0
GET	/metrics/stats [heart_rate]	440	0	96	130	210	101.43	82	367	144.11	3.9	0
GET	/strain	112	0	94	110	210	99.02	84	347	224.46	1.5	0
	Aggregated	8482	0	91	110	210	94.84	77	1206	227.33	72.7	0

Figure 2a. Locust statistics of 2 ECS task; 100 users, 10 spawn rate, 2 minutes.

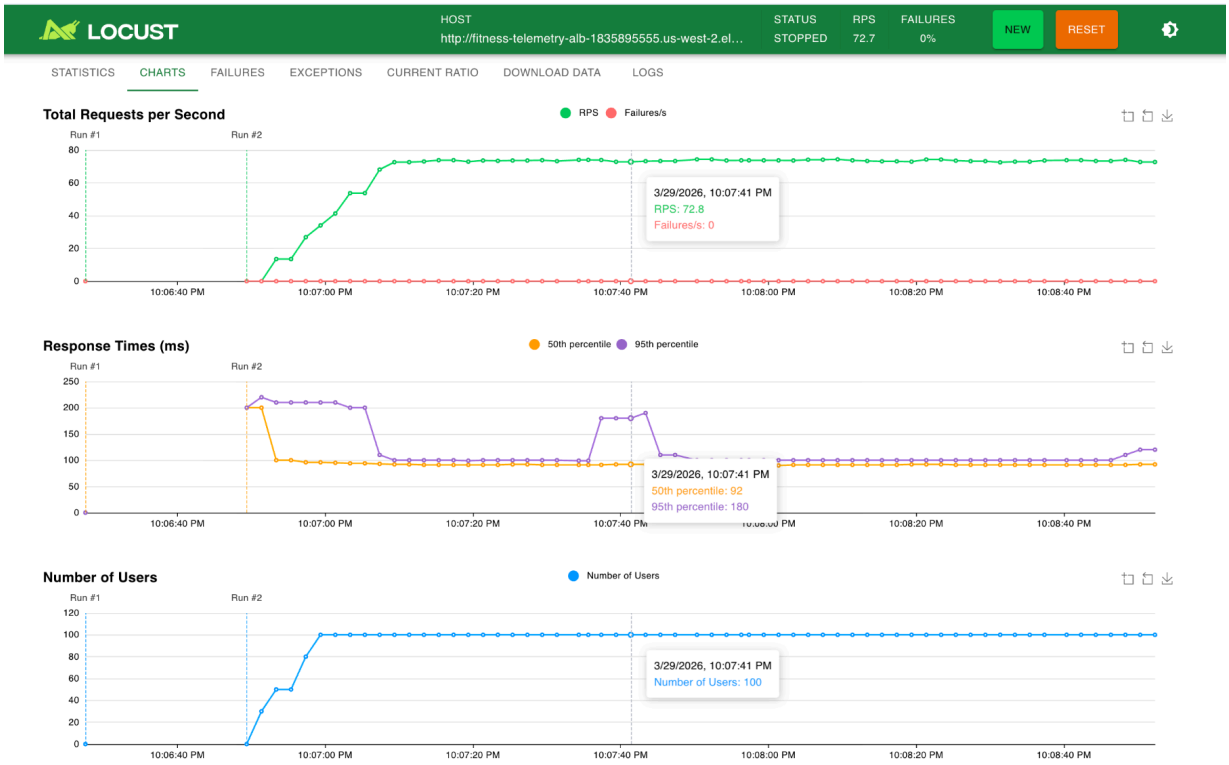
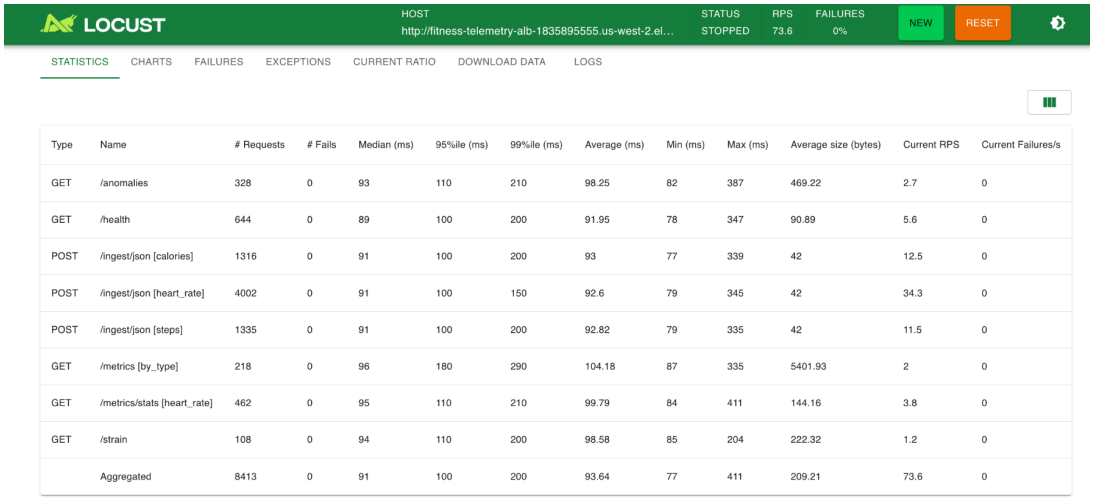


Figure 2b. Locust graphs of 2 ECS task; 100 users, 10 spawn rate, 2 minutes.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/anomalies	328	0	93	110	210	98.25	82	387	469.22	2.7	0
GET	/health	644	0	89	100	200	91.95	78	347	90.89	5.6	0
POST	/ingest/json [calories]	1316	0	91	100	200	93	77	339	42	12.5	0
POST	/ingest/json [heart_rate]	4002	0	91	100	150	92.6	79	345	42	34.3	0
POST	/ingest/json [steps]	1335	0	91	100	200	92.82	79	335	42	11.5	0
GET	/metrics [by_type]	218	0	96	180	290	104.18	87	335	5401.93	2	0
GET	/metrics/stats [heart_rate]	462	0	95	110	210	99.79	84	411	144.16	3.8	0
GET	/strain	108	0	94	110	200	98.58	85	204	222.32	1.2	0
	Aggregated	8413	0	91	100	200	93.64	77	411	209.21	73.6	0

Figure 3a. Locust statistics of 4 ECS task; 100 users, 10 spawn rate, 2 minutes.

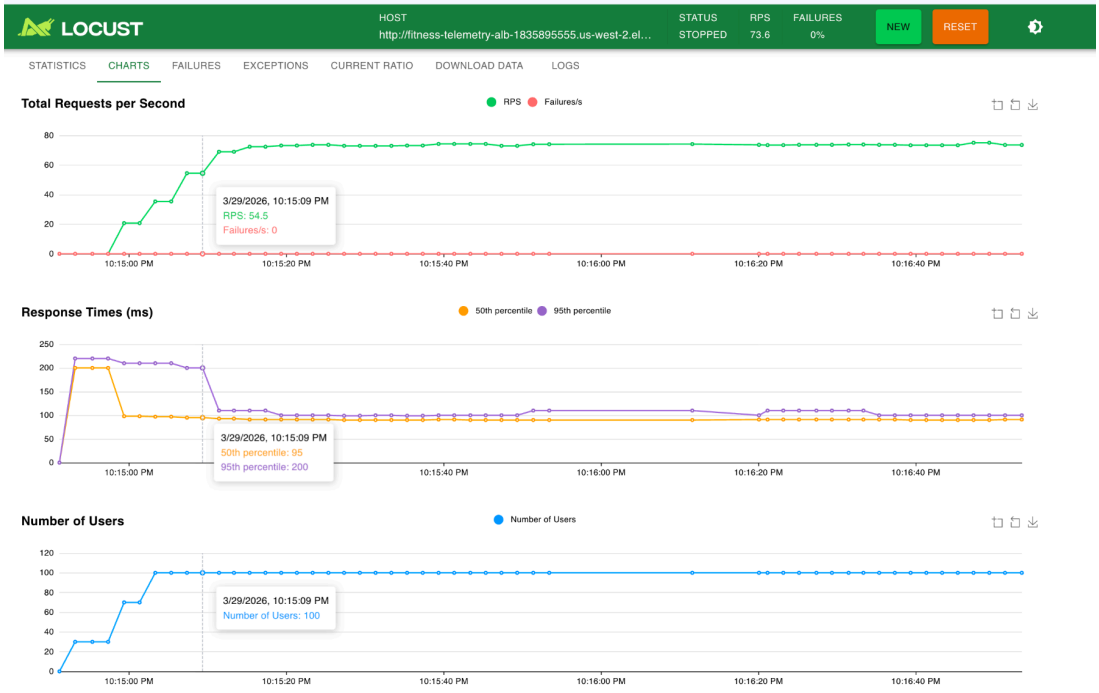


Figure 3b. Locust graphs of 4 ECS task; 100 users, 10 spawn rate, 2 minutes.

Across all three runs, the system demonstrated strong baseline stability with zero failures regardless of task count. As shown in Figure 1 (1 ECS task), the server sustained approximately 72 requests per second at a P95 latency of 100ms and P99 of 170ms under 100 concurrent users. Scaling to 2 tasks (Figure 2) and 4 tasks (Figure 3) produced no meaningful latency improvement — P95 held at 100–110ms and P99 actually rose slightly to 210ms — suggesting that at this user count the single-task baseline was not the bottleneck. The ingest endpoints consistently outperformed read endpoints, with /ingest/json averaging 91–92ms against /metrics averaging 100–105ms, reflecting the higher computation involved in querying and serializing accumulated records. The flat throughput curve across task counts points to the in-memory store as the likely ceiling: because each ECS task maintains its own independent state, scaling horizontally does not reduce per-task work or pool shared memory, meaning additional tasks add routing overhead without relieving the actual load. To identify the true horizontal scaling ceiling, a follow-up run at 500+ concurrent users would be needed to saturate the single-task baseline and expose where ECS task count begins to matter.

Test 2:

Operation	Count	Avg (ms)	Min (ms)	Max (ms)	P50 (ms)	P95 (ms)	P99 (ms)	Success
Ingest (/ingest/json)	50	184.0	176	201	184	192	201	100%
Query (/metrics?limit=200)	50	185.1	174	199	184	195	199	100%
Stats (/metrics/stats)	50	293.7	175	701	189	570	701	100%
Aggregated	150	220.9	174	701	184	520	680	100%

All 150 operations succeeded with zero failures. Ingest and query operations were tightly consistent, averaging 184ms and 185ms respectively with P99 latencies of 201ms and 199ms, reflecting the low overhead of writing to and reading from an in-process sync.RWMutex slice with no network round trips to an external store. Stats queries showed higher variance, averaging 294ms with a P99 of 701ms, attributable to the fan-out across all metric types and the volume of accumulated records being aggregated. These results represent the theoretical performance ceiling of the system — any durable persistence layer (DynamoDB or MySQL) would add write latency on top of this baseline. Given the tight P99 on ingest and the stateless compute model, DynamoDB on-demand is the recommended persistence path for a future implementation, as its partition-key-per-user access pattern aligns directly with the time-series workload and would add an estimated 5–15ms per write at low-to-moderate throughput, preserving the sub-200ms ingest latency budget.

Test 3:

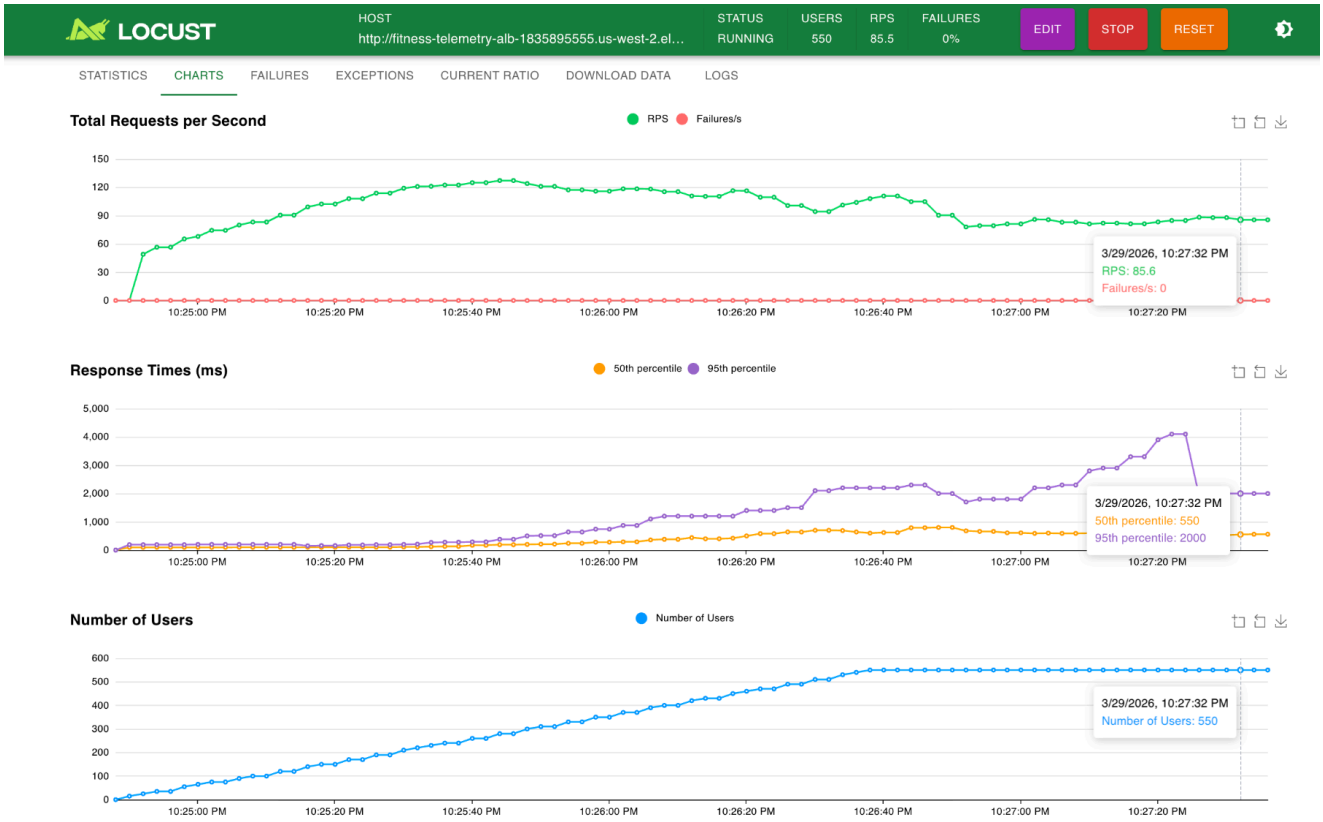


Figure 4. Locust graph before chaos injectoion

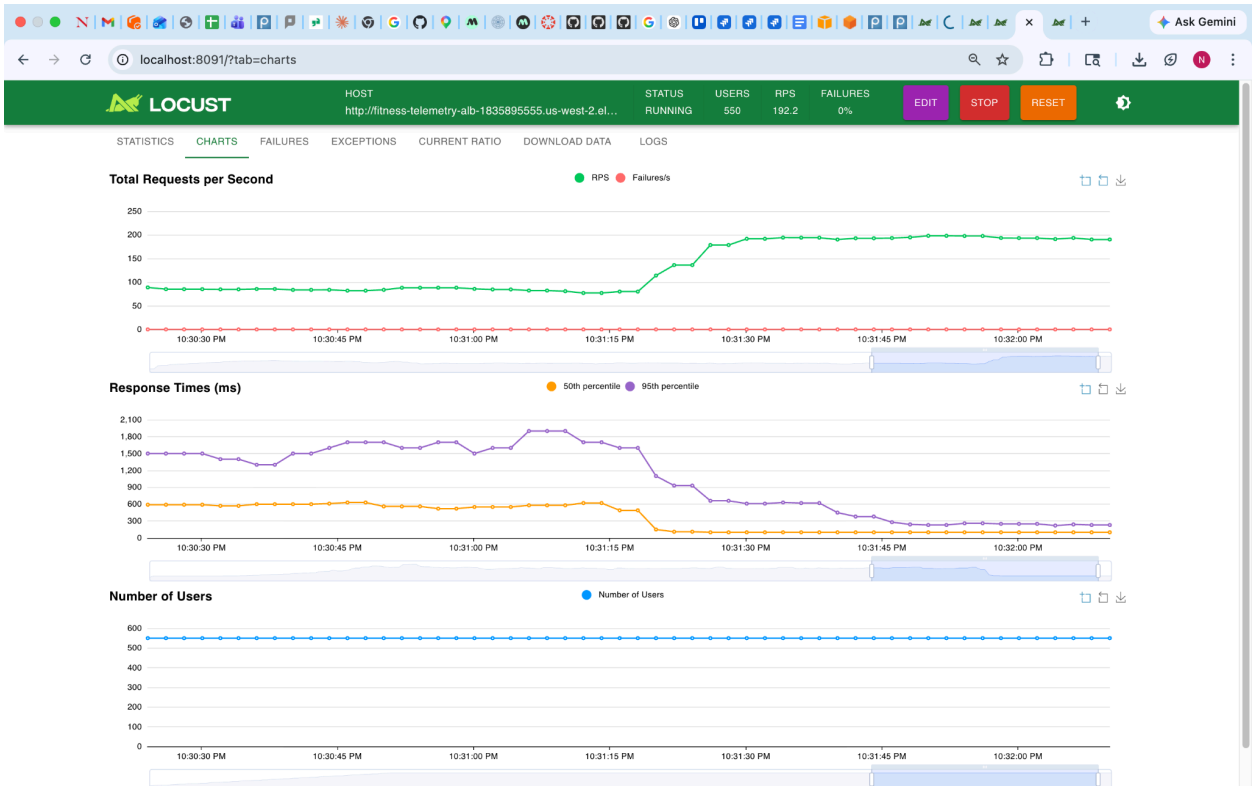
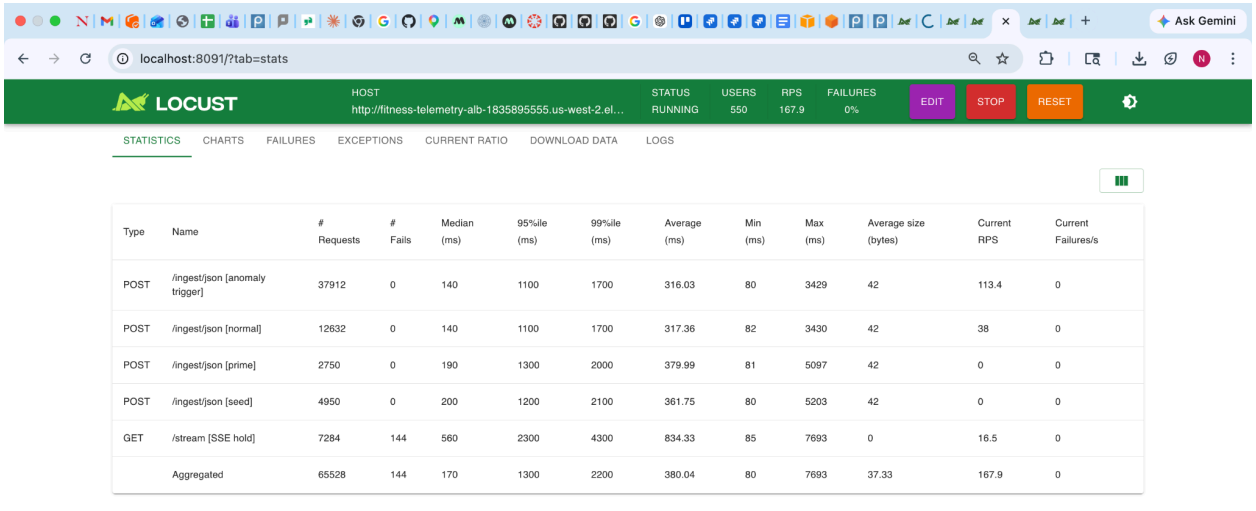


Figure 5. Locust graph after chaos injection



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/ingest/json [anomaly trigger]	37912	0	140	1100	1700	316.03	80	3429	42	113.4	0
POST	/ingest/json [normal]	12632	0	140	1100	1700	317.36	82	3430	42	38	0
POST	/ingest/json [prime]	2750	0	190	1300	2000	379.99	81	5097	42	0	0
POST	/ingest/json [seed]	4950	0	200	1200	2100	361.75	80	5203	42	0	0
GET	/stream [SSE hold]	7284	144	560	2300	4300	834.33	85	7693	0	16.5	0
	Aggregated	65528	144	170	1300	2200	380.04	80	7693	37.33	167.9	0

Figure 6. Failures notices in API calls after chaos injection

Experiment 3 stress-tested the SSE fan-out layer under 550 concurrent users — 90% holding persistent /stream connections for 20–40 seconds while 10% continuously injected anomaly-triggering heart rate readings (values of 5–10 bpm and 220–230 bpm, well outside the normal 58–82 bpm window, guaranteed to trigger z-score alerts and push events to all connected clients). The pre-injection snapshot (Figure 4, 4 minutes) recorded zero failures with SSE connections averaging 1,101ms and peaking at P99 6,600ms, reflecting the cost of spawning 495 concurrent long-lived connections against a single ECS task. Over the full 8-minute 42-second run (Figure 5), the system processed 60,779 total requests and stabilised — ingest latency dropped from an average of 526ms to 332ms and SSE average fell from 1,101ms to 868ms as the rolling window warmed and the anomaly detector reached steady state. The 144 SSE failures (2.1% of stream requests) were exclusively ALB read timeouts, occurring when connections exceeded the ALB's 60-second idle timeout threshold mid-stream rather than due to server-side crashes or dropped goroutines — evidenced by the fact that failures occurred after widely varying event counts (25 to 1,633 events) with no clustering pattern. Critically, the ingest path recorded zero failures throughout, demonstrating that the SSE broadcast layer degrades gracefully under fan-out pressure without contaminating the write path. The primary bottleneck identified is the ALB idle timeout configuration, not ECS connection handling capacity — increasing the ALB timeout to 300 seconds would eliminate the majority of observed failures in a production deployment.